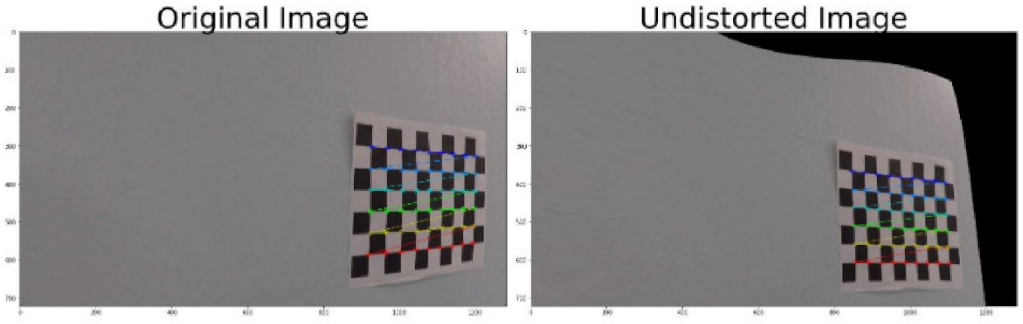
Advanced Lane Finding WRITE UP

The writeup / README includes a statement and supporting figures / images that explain how each rubric item was addressed, and specifically where in the code each step was handled.

1. Use chessboard image to compute calibration



Camera lenses can distort images which is why we must start with camera calibration which is obtained by calculating distortion coefficients.

We use chessboard images provided in the camera_cal folder because of their high contrast and squares that are all equal.

```
objp = np.zeros((6*9,3), np.float32)
objp[:,:2] = np.mgrid[0:9,0:6].T.reshape(-1,2)
```

There are 9 * 6 corners to work with.

The object points will have x and y coordinates generated from grid indexes. z is always 0.

The image points represent the corresponding object points found in the image using OpenCV's function findChessboardCorners.

```
# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Find the chessboard corners
nx = 9
ny = 6
ret, corners = cv2.findChessboardCorners(gray, (nx, ny), None)
```

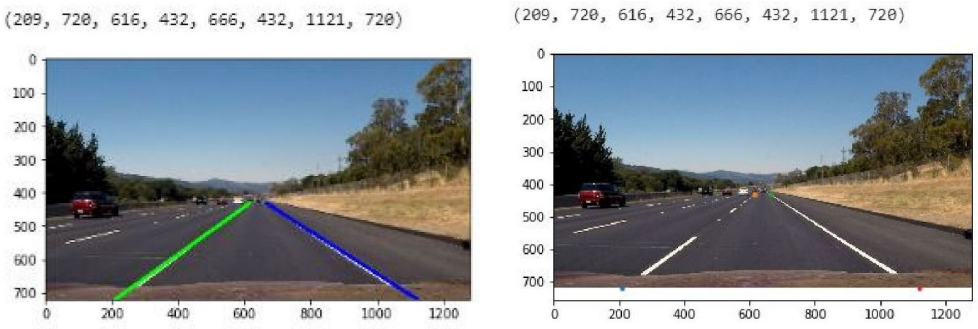
The calibrateCamera function is used to get the camera matrix and distortion coefficients identification after all images are scanned.
The OpenCV undistort function transforms the images using the camera matrix and distortion coefficients.

```
ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(objpoints, imgpoints, gray.shape[::-1], None, None)
undist = cv2.undistort(img, mtx, dist, None, mtx)
```

As can be seen in the image below, the distorted image is different from the original image. It is particularly visible around the edge, for instance the white car, and in the sligh curve change of the road.

2. Perspective transform. Bird's eye view

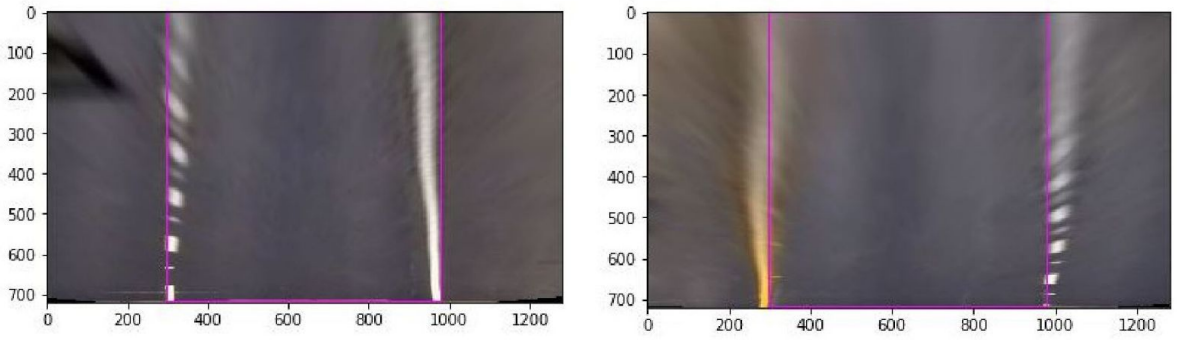
We use the fact that the lanes are parallel. We first identity source and destination points from the image of the perspective transform.



```
# Source points taken from images with straight lane lines,
# After the warp transform the lane lines are parallel
src = np.float32([
    (190, 720), # bottom-left corner
    (596, 447), # top-left corner
    (685, 447), # top-right corner
    (1125, 720) # bottom-right corner
])

# Destination points are to be parallel
# Account for the image size
dst = np.float32([
    [offset, img_size[1]],             # bottom-left corner
    [offset, 0],                       # top-left corner
    [img_size[0]-offset, 0],           # top-right corner
    [img_size[0]-offset, img_size[1]]  # bottom-right corner
])
```

We use the warpPerspective function to perform the bird's eye view perspective transform.

```
# Calculate the transformation matrix and it's inverse transformation
M = cv2.getPerspectiveTransform(src, dst)
M_inv = cv2.getPerspectiveTransform(dst, src)
warped = cv2.warpPerspective(undist, M, img_size)
```



3. Process Binary Thresholded Images

This process is obtained through 4 different transformations.

- Sobel: We take the x sobel on the gray-scaled image. This represents the derivative in the x direction and helps detect lines that tend to be vertical. Only the values above a minimum threshold are kept.

```
# Transform image to gray scale
gray_img =cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply sobel (derivative) in x direction, this is usefull to detect lines that tend to be vertical
sobelx = cv2.Sobel(gray_img, cv2.CV_64F, 1, 0)
abs_sobelx = np.absolute(sobelx)

# Scale result to 0-255
scaled_sobel = np.uint8(255*abs_sobelx/np.max(abs_sobelx))
sx_binary = np.zeros_like(scaled_sobel)

# Keep only derivative values that are in the margin of interest
sx_binary[(scaled_sobel >= 30) & (scaled_sobel <= 255)] = 1
```

- White pixels: We detect pixels that are white in the grayscale image

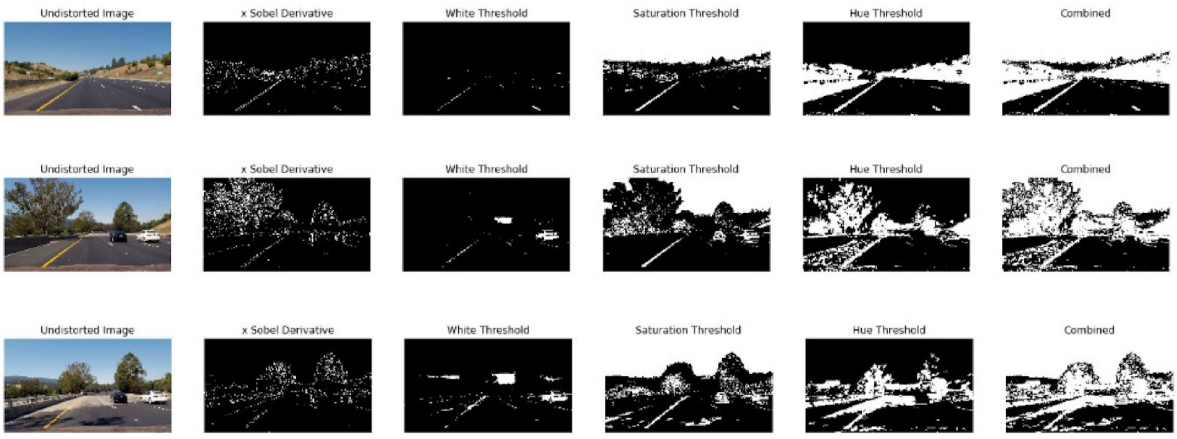
```
# Detect pixels that are white in the grayscale image
white_binary = np.zeros_like(gray_img)
white_binary[(gray_img > 200) & (gray_img <= 255)] = 1
```

- HSL color space: we first convert the image then we detect pixels that have a high saturated value. This is crucial for the yellow lines.

```
# Convert image to HLS
hls = cv2.cvtColor(img, cv2.COLOR_BGR2HLS)
H = hls[:,:,0]
S = hls[:,:,2]
sat_binary = np.zeros_like(S)

# Detect pixels that have a high saturation value
sat_binary[(S > 90) & (S <= 255)] = 1
```

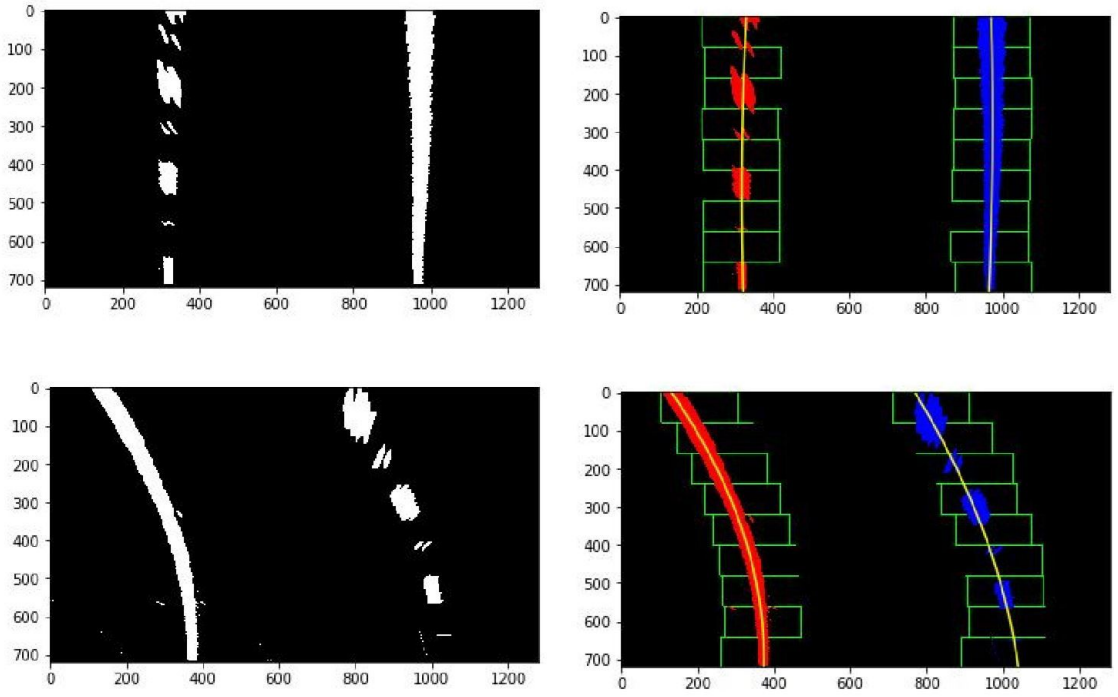
- Hue : use the hue to detect yellow pixels. (between 10 and 25)

```
hue_binary =  np.zeros_like(H)

# Detect pixels that are yellow using the hue component
hue_binary[(H > 10) & (H <= 25)] = 1
```



4. Lane Line Detection Using Histogram

A histogram is computed on the unwarped undistorted images.
Pixel values are summed on each column to detect the most probable x position of left and right lane lines.



```
# Histogram of the bottom half of the image
histogram = np.sum(binary_warped[binary_warped.shape[0]//2:,:], axis=0)

# Find peak of the left and right halves of the histogram
# These are the starting point for the left and right lines
midpoint = np.int(histogram.shape[0]//2)
leftx_base = np.argmax(histogram[:midpoint])
rightx_base = np.argmax(histogram[midpoint:]) + midpoint
```

We use the sliding window to search for line pixel starting from the bottom of the image and going toward the top. The average position of coordinates of lane pixels within the window is computed if enough pixels are found.

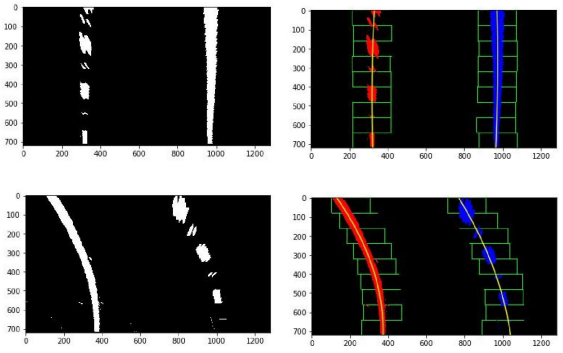
.

```
# Choose number of sliding windows
nwindows = 9

# Set width of the windows +/- margin
margin = 100

# Set minimum number of pixels found to re-center window
```

```
minpix = 50

# Identify window boundaries in x and y (and right and left)
win_y_low = binary_warped.shape[0] - (window+1)*window_height
win_y_high = binary_warped.shape[0] - window*window_height
win_xleft_low = leftx_current - margin
win_xleft_high = leftx_current + margin
win_xright_low = rightx_current - margin
win_xright_high = rightx_current + margin

# Identify nonzero pixels in x and y within the window #
good_left_inds = ((nonzeroy >= win_y_low) & (nonzeroy < win_y_high) &
(nonzerox >= win_xleft_low) &  (nonzerox < win_xleft_high)).nonzero()[0]
good_right_inds = ((nonzeroy >= win_y_low) & (nonzeroy < win_y_high) &
(nonzerox >= win_xright_low) &  (nonzerox < win_xright_high)).nonzero()[0]

# Append these indices to the lists
left_lane_inds.append(good_left_inds)
right_lane_inds.append(good_right_inds)
```

For both lanes the pixels get appended to the list of their x and y coordinates.
Then a polynomial is fitted on each to find the best fit of the selected pixels.

```
# Fit second degree polynomial to each with np.polyfit()
left_fit = np.polyfit(lefty, leftx, 2)
right_fit = np.polyfit(righty, rightx, 2)
```



We can clearly see the red and blue lines.

5. Detection of Lane Lines Based on Previous Cycle

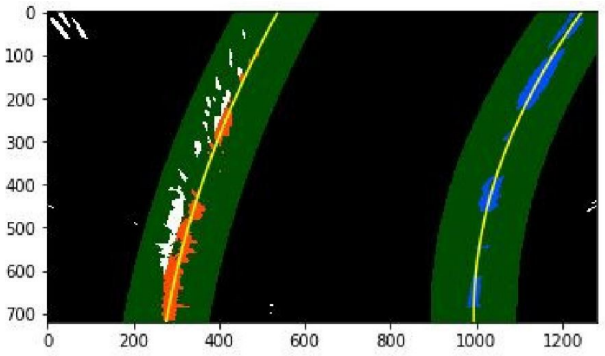
.
We know that the lanelines will be close to the ones from the previous cycle.

We use the data obtained from the previous cycle in order to get a faster result

We use a sliding window method along the polynomial fit for the left and right line from the previous cycle.

```
# Set area of search based on activated x-values within the +/- margin of our polynomial
function
left_lane_inds = ((nonzerox > (prev_left_fit[0]*(nonzeroy**2) + prev_left_fit[1]*nonzeroy +
            prev_left_fit[2] - margin)) & (nonzerox < (prev_left_fit[0]*(nonzeroy**2) +
            prev_left_fit[1]*nonzeroy + prev_left_fit[2] + margin))).nonzero()[0]
right_lane_inds = ((nonzerox > (prev_right_fit[0]*(nonzeroy**2) +
prev_right_fit[1]*nonzeroy +
            prev_right_fit[2] - margin)) & (nonzerox < (prev_right_fit[0]*(nonzeroy**2) +
            prev_right_fit[1]*nonzeroy + prev_right_fit[2] + margin))).nonzero()[0]

# Again, extract left and right line pixel positions
leftx = nonzerox[left_lane_inds]
lefty = nonzeroy[left_lane_inds]
rightx = nonzerox[right_lane_inds]
righty = nonzeroy[right_lane_inds]
```

The leftx, lefty, rightx, righty pixel coordinates are fitted with a second degree polynomial function for each left and right side.



6. Calculate Vehicle Position and Curve Radius

We first use scaling factors.

- 30 meters to 720 pixels in the y direction
- 3.7 meters to 700 pixels in the x dimension.

We then use a polynomial fit to make the conversion . We use the coordinates of the aligned pixels from the fitted line.

The radius of the curvature is calculated with the y point at the bottom of the image.

```
# Define conversions in x and y from pixels space to meters
ym_per_pix = 30/720 # meters per pixel in y dimension
xm_per_pix = 3.7/700 # meters per pixel in x dimension

left_fit_cr = np.polyfit(ploty*ym_per_pix, left_fitx*xm_per_pix, 2)
right_fit_cr = np.polyfit(ploty*ym_per_pix, right_fitx*xm_per_pix, 2)

# Define y-value where we want radius of curvature
# We'll choose the maximum y-value, corresponding to the bottom of the image
y_eval = np.max(ploty)

# Calculation of R_curve (radius of curvature)
left_curverad = ((1 + (2*left_fit_cr[0]*y_eval*ym_per_pix + left_fit_cr[1])**2)**1.5) /
np.absolute(2*left_fit_cr[0])
right_curverad = ((1 + (2*right_fit_cr[0]*y_eval*ym_per_pix + right_fit_cr[1])**2)**1.5) /
np.absolute(2*right_fit_cr[0])
```

The vehicle's position is obtained with the polynomial fit: we determine the x position of the left and right lane corresponding to the y at the bottom of the image.

```
# Define conversion in x from pixels space to meters
xm_per_pix = 3.7/700 # meters per pixel in x dimension

# Choose the y value corresponding to the bottom of the image
y_max = binary_warped.shape[0]

# Calculate left and right line positions at the bottom of the image
left_x_pos = left_fit[0]*y_max**2 + left_fit[1]*y_max + left_fit[2]
right_x_pos = right_fit[0]*y_max**2 + right_fit[1]*y_max + right_fit[2]

# Calculate the x position of the center of the lane
center_lanes_x_pos = (left_x_pos + right_x_pos)//2

# Calculate the deviation between the center of the lane and the center of the picture
```

```
# The car is assumed to be placed in the center of the picture
# If the deviation is negative, the car is on the felt hand side of the center of the lane
veh_pos = ((binary_warped.shape[1]//2) - center_lanes_x_pos) * xm_per_pix
```

The position of the center of the lane in the image is obtained with the average of these two values.

Since the camera is mounted on the central axis of the vehicle iIf the lane's center is shifted to the right by nbp amount of pixels that means that the car is shifted to the left by nbp * xm_per_pix meters.

Screenshot from the processed video.

7. Consideration of problems that could be improved about the algorithm/pipeline, and what hypothetical cases would cause pipeline to fail.

The problems mostly came from lighting conditions and shadows.
When playing around with the normalization technique of the images, I realized that some normalization technique can make more noise. It can be hard to choose the right one that matches the weather. A solution could be adding a weather detector that then chooses the appropriate normalization for the images at the beginning of the pipeline.